

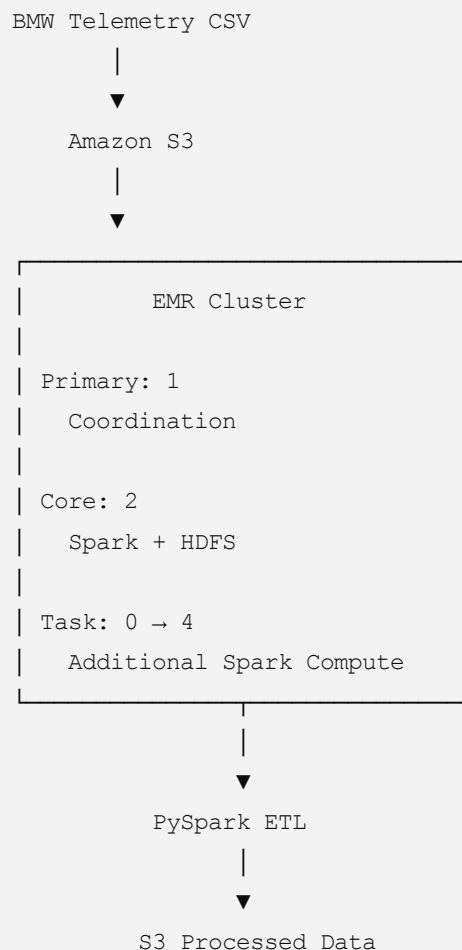
Worked for 23s

Below is a complete **hands-on EMR on EC2 lab** covering exactly these objectives:

- Design **Primary, Core, Task** nodes
- Choose a practical cluster size
- Configure **EMR Managed Scaling**
- Run a **BMW PySpark job**
- Verify scaling and output

AWS supports EMR clusters built with instance groups or instance fleets; for this first lab, **instance groups are easier to understand**. [AWS Document... +1](#)

Lab architecture



For this lab, use **US East (N. Virginia)** `us-east-1` if that is where your existing AWS resources are.

Step 1 — Create the S3 folders

Create an S3 bucket such as:

bmw-emr-lab-2026

Create this structure:

```
bmw-emr-lab-2026/  
|  
├─ raw/  
|   └─ telemetry/  
|  
├─ scripts/  
|  
├─ processed/  
|  
└─ logs/
```

Your architecture starts as:

```
S3  
├─ raw          input  
├─ scripts      PySpark programs  
├─ processed    EMR output  
└─ logs         EMR logs
```

Step 2 — Create sample BMW telemetry

Create:

bmw_telemetry.csv

with:

CSV

```
vehicle_id,model,battery_level,temperature,speed,region  
BMW1001,iX,18,92,75,Chennai  
BMW1002,i4,65,78,85,Bangalore  
BMW1003,iX,42,86,80,Chennai  
BMW1004,X5,35,96,90,Mumbai  
BMW1005,i4,15,83,70,Delhi  
BMW1006,iX,75,80,95,Bangalore  
BMW1007,X3,55,89,82,Chennai  
BMW1008,X5,28,94,88,Mumbai
```

Upload it to:

s3://bmw-emr-lab-2026/raw/telemetry/

Step 3 — Create the PySpark program

Create:

bmw_emr_job.py

Use:

Python

Run

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, count

spark = SparkSession.builder \
    .appName("BMWTelemetryEMR") \
    .getOrCreate()

input_path = "s3://bmw-emr-lab-2026/raw/telemetry/"
output_path = "s3://bmw-emr-lab-2026/processed/telemetry/"

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(input_path)

print("BMW TELEMETRY")
df.show()

print("LOW BATTERY VEHICLES")

low_battery = df.filter(
    df.battery_level < 20
)

low_battery.show()

print("HIGH TEMPERATURE VEHICLES")

high_temperature = df.filter(
    df.temperature > 90
)
```

```

high_temperature.show()

print("MODEL ANALYSIS")

result = df.groupBy("model").agg(

    count("vehicle_id").alias("vehicle_count"),

    avg("battery_level").alias(
        "average_battery"
    ),

    avg("temperature").alias(
        "average_temperature"
    )

)

result.show()

result.write \
    .mode("overwrite") \
    .parquet(output_path)

spark.stop()

```

Upload it to:

```
s3://bmw-emr-lab-2026/scripts/bmw_emr_job.py
```

Step 4 — Make sure EMR IAM roles exist

EMR requires:

```

EMR Service Role
+
EC2 Instance Profile Role

```

The current EMR console does not necessarily create default IAM roles automatically, so make sure suitable roles exist before launching the cluster. [AWS Document... +1](#)

For a training account, you can create the default EMR roles using AWS CLI:

Bash

```
aws emr create-default-roles
```

AWS documents this command as creating:

```
EMR_DefaultRole
```

```
EMR_EC2_DefaultRole
```

 AWS Documen... +1

Verify:

IAM → Roles

Search for:

```
EMR_DefaultRole
```

```
EMR_EC2_DefaultRole
```

Step 5 — Give the EMR EC2 role access to your S3 bucket

Your EMR nodes need to read:

```
raw/  
scripts/
```

and write:

```
processed/  
logs/
```

For a lab, add an inline policy to the EMR EC2 role.

Go to:

IAM → Roles → EMR_EC2_DefaultRole → Add permissions → Create inline policy

Use:

```
JSON
```

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::bmw-emr-lab-2026"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::bmw-emr-lab-2026/*"
      ]
    }
  ]
}
```

Replace the bucket name with yours.

Step 6 — Create the EMR cluster

Go to:

AWS Console → Amazon EMR

Then:

EMR on EC2 → Clusters → Create cluster

AWS currently recommends using a recent EMR release for managed scaling; current documentation recommends EMR 7.13.0 where available. [AWS Document...](#)

Use:

Cluster name:
bmw-emr-cluster

Amazon EMR release:
Latest available EMR 7.x

Applications:
Spark

You can optionally add:

Hadoop
Hive

For this lab, **Spark is sufficient**.

Step 7 — Choose Instance Groups

Look for:

Cluster configuration / Hardware configuration

Choose:

Uniform instance groups

Do not select instance fleets for the first lab.

The node structure will be:

Primary Instance Group

Core Instance Group

Task Instance Group

With instance groups, each group uses instances of the same EC2 type.  AWS Documen...

Step 8 — Configure the Primary node

Configure:

Node type:
Primary

Instance count:
1

Instance type:
m5.xlarge

Purchasing option:
On-Demand

Architecture:

Primary
m5.xlarge
On-Demand

The Primary node manages the cluster, tracks jobs, runs components such as YARN ResourceManager and HDFS NameNode, and monitors cluster health. [AWS Documen...](#)

Step 9 — Configure Core nodes

Configure:

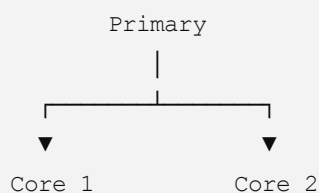
Node type:
Core

Instance count:
2

Instance type:
m5.xlarge

Purchasing option:
On-Demand

You now have:



Core nodes:

Process Spark jobs
+
Store HDFS data

[AWS Documen...](#)

Step 10 — Configure Task nodes

For the initial lab, you can either create one Task node or allow managed scaling to add Task capacity.

For a visually clearer demonstration, start with:

Task nodes:

1

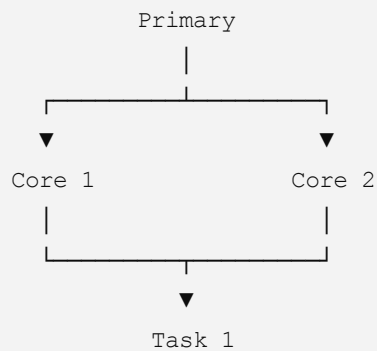
Instance type:

m5.xlarge

Purchasing:

On-Demand

Initial cluster:



Task nodes run Spark executors and other processing tasks but don't store HDFS data.

 AWS Documen...

Step 11 — Configure networking

Under:

Networking

Select your VPC.

For a basic lab:

VPC:

Default VPC

Subnet:

A subnet in us-east-1

For simple training, using the default VPC is usually easier.

You don't need SSH for this lab because we're submitting the Spark script as an EMR Step.

Step 12 — Configure logging

Enable:

```
Publish cluster logs to Amazon S3
```

Enter:

```
s3://bmw-emr-lab-2026/logs/
```

AWS recommends using an S3 log path if you want cluster logs retained.  [AWS Document...](#)

Step 13 — Select IAM roles

Under:

Security configuration and permissions

Select:

```
Service role:  
EMR_DefaultRole
```

and:

```
IAM role for instance profile:  
EMR_EC2_DefaultRole
```

If you created custom EMR roles, select those instead.

Step 14 — Configure Managed Scaling

This is the important part of the lab.

Find:

Cluster scaling and provisioning option

Select:

Use EMR-managed scaling

AWS Managed Scaling automatically adjusts Core and Task capacity according to workload. The Primary node is not scaled.  AWS Documen... +1

For our BMW lab enter:

Minimum capacity:

2

Maximum capacity:

6

Maximum core capacity:

2

Maximum On-Demand capacity:

6

Interpretation:

Minimum = 2

means keep at least the equivalent of two worker instances.

Maximum = 6

means Core + Task processing capacity cannot exceed six instances.

Maximum Core = 2

means:

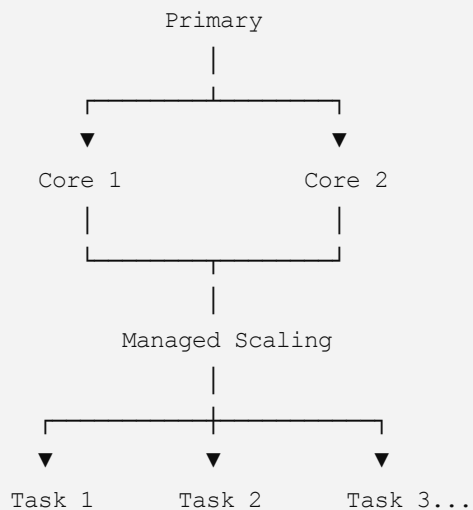
Core stays around 2

while additional processing capacity goes to Task nodes.

So:

Core = 2 fixed baseline

Conceptually:



AWS defines these scaling boundaries as applying to **Core and Task**, not the Primary node. [AWS Document...](#)

Step 15 — Create the cluster

Click:

Create cluster

You'll see cluster states such as:

```
STARTING
  ↓
BOOTSTRAPPING
  ↓
RUNNING
  ↓
WAITING
```

Wait until:

```
Status = Waiting
```

or:

```
Running
```

depending on the workload/state.

Step 16 — Verify the nodes

Open:

EMR → **Clusters** → **bmw-emr-cluster** → **Instances**

You should see approximately:

```
Primary
1 × m5.xlarge

Core
2 × m5.xlarge

Task
0/1+ × m5.xlarge
```

You can inspect:

```
Instance ID
Node type
Instance type
Market
Status
```

Step 17 — Submit the BMW Spark job

Go to:

Cluster → **Steps** → **Add step**

Choose:

```
Step type:
Spark application
```

If the console presents **Custom JAR** instead, you can use `command-runner.jar`, but Spark application is preferable when available.

Enter:

```
Name:
BMW Telemetry PySpark ETL
```

Application location:

```
s3://bmw-emr-lab-2026/scripts/bmw_emr_job.py
```

Then click:

Add step

Step 18 — Watch job status

You should see:

```
Pending
  ↓
Running
  ↓
Completed
```

If it fails:

```
Failed
```

Open the Step details and inspect the logs.

Step 19 — Check the Spark output

Go to:

Amazon S3 → bmw-emr-lab-2026 → processed → telemetry

You should see files similar to:

```
_SUCCESS

part-00000-....snappy.parquet

part-00001-....snappy.parquet
```

This proves:

```
S3 input
  ↓
EMR Spark
  ↓
Processing
```

↓
Parquet output

Step 20 — Check the expected analysis

Your Spark job should identify low-battery vehicles such as:

```
BMW1001    iX    18
BMW1005    i4    15
```

and high-temperature vehicles such as:

```
BMW1001    92
BMW1004    96
BMW1008    94
```

It also calculates:

```
Model
Vehicle Count
Average Battery
Average Temperature
```

Step 21 — Observe Managed Scaling

Go to:

EMR → Cluster → Instances

During a heavier workload, EMR can increase worker capacity:

```
Before
-----
Core = 2
Task = 0/1
```

then:

```
During heavy Spark job
-----
Core = 2
Task = 2-4
```

and after demand drops:

```
After workload
```

```
-----
```

```
Core = 2
```

```
Task = 0
```

Managed scaling continually evaluates workload metrics and adjusts capacity within the minimum and maximum limits you've configured. [AWS Documen...](#)

With only eight CSV rows, however, **you probably won't observe a scale-out** because there isn't enough work.

Step 22 — Create enough workload to observe autoscaling

For an autoscaling demonstration, generate a larger dataset.

You could create:

```
1 million+
```

BMW telemetry rows and upload them to:

```
s3://bmw-emr-lab-2026/raw/telemetry-large/
```

Then run transformations involving:

```
groupBy  
join  
orderBy  
repartition  
aggregation
```

For example:

Python

Run

```
large_df = df.repartition(200)  
  
large_df.groupBy(  
    "model",  
    "region"  
) .avg(  
    "battery_level",
```


More pending Spark/YARN work gives Managed Scaling something to react to.

Step 23 — Check the scaling configuration

From your cluster:

Instances → **Instance group settings** → **Edit cluster scaling**

You should see:

```
Minimum:
2

Maximum:
6

Maximum Core:
2

Maximum On-Demand:
6
```

AWS supports modifying managed-scaling settings on a running cluster from the Instances page. [AWS Documen...](#)

Step 24 — Verify Managed Scaling from CLI

Get your cluster ID:

Bash

```
aws emr list-clusters ^
--active ^
--region us-east-1
```

Suppose:

```
j-ABCDEFGHIJK
```

Run:

Bash

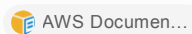
```
aws emr get-managed-scaling-policy ^  
--cluster-id j-ABCDEFGHIJK ^  
--region us-east-1
```

Expected structure:

JSON

```
{  
  "ManagedScalingPolicy": {  
    "ComputeLimits": {  
      "MinimumCapacityUnits": 2,  
      "MaximumCapacityUnits": 6,  
      "MaximumOnDemandCapacityUnits": 6,  
      "MaximumCoreCapacityUnits": 2,  
      "UnitType": "Instances"  
    }  
  }  
}
```

AWS provides `get-managed-scaling-policy` specifically to inspect the active policy.



Step 25 — Modify autoscaling using CLI

You can also change it programmatically:

Bash

```
aws emr put-managed-scaling-policy ^  
--cluster-id j-ABCDEFGHIJK ^  
--managed-scaling-policy "ComputeLimits={MinimumCapacityUnits=2,MaximumCapacityUnits=8,MaximumOnDemandCapacityUnits=8,MaximumCoreCapacityUnits=2,UnitType=Instances}"  
--region us-east-1
```

Now your range becomes:

Core baseline = 2

Maximum worker capacity = 8

Potential Task capacity = up to 6

Step 26 — Cluster sizing exercise

Run the same job with different cluster sizes.

Test A — Small

```
Primary = 1
Core = 2
Task = 0
```

Record:

```
Job duration
CPU
Memory
Cost
```

Test B — Medium

```
Primary = 1
Core = 2
Task = 2
```

Test C — Larger

```
Primary = 1
Core = 2
Task = 4
```

Then compare:

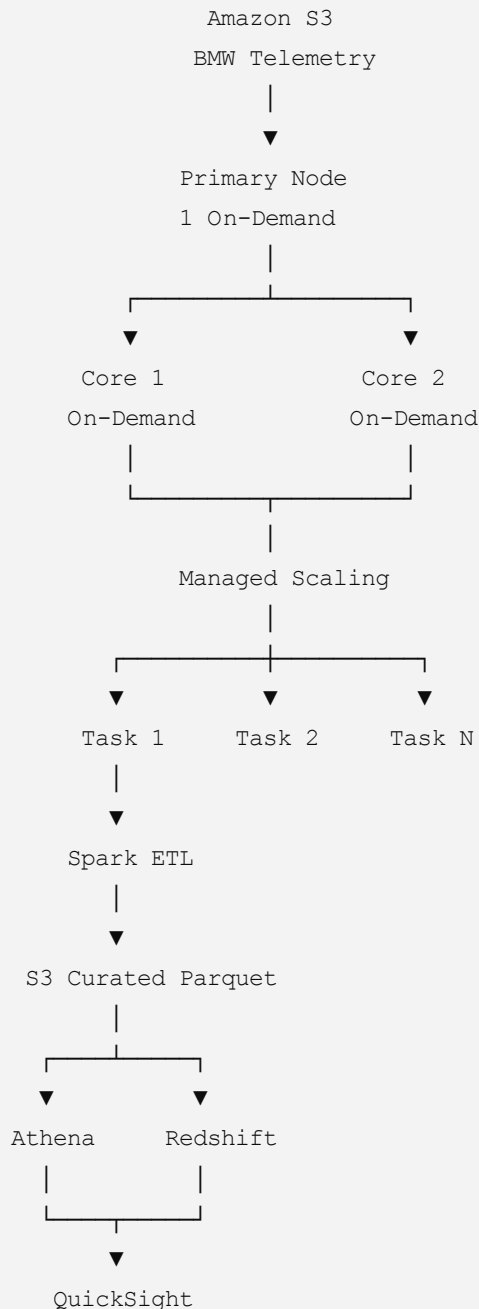
Design	Primary	Core	Task	Expected use
Development	1	2	0	Small jobs
Medium	1	2	2	Normal ETL
Peak	1	2	4	Heavy ETL

AWS's sizing guidance recommends benchmarking with representative data and monitoring utilization rather than selecting cluster size only from dataset size.

 AWS Document...

Step 27 — Practical BMW production design

Once the basic lab works, a stronger architecture is:



For production, **Task nodes** are particularly useful for elastic compute because they **do not store HDFS data**. [AWS Document...](#)

Important final step — terminate the cluster

EMR on EC2 costs money while the EC2 resources remain running.

After the lab:

Amazon EMR → Clusters → bmw-emr-cluster → Terminate

Confirm termination.

Your S3 data remains:

```
raw/  
scripts/  
processed/  
logs/
```

while the EC2-based EMR cluster is removed.

Recommended lab configuration

```
Cluster:  
bmw-emr-cluster  
  
Release:  
Latest EMR 7.x  
  
Applications:  
Spark  
  
Primary:  
1 × m5.xlarge On-Demand  
  
Core:  
2 × m5.xlarge On-Demand  
  
Managed Scaling:  
Minimum = 2  
Maximum = 6  
Maximum Core = 2  
Maximum On-Demand = 6  
  
Input:  
s3://bmw-emr-lab-2026/raw/telemetry/  
  
PySpark:  
s3://bmw-emr-lab-2026/scripts/bmw_emr_job.py  
  
Output:  
s3://bmw-emr-lab-2026/processed/telemetry/  
  
Logs:  
s3://bmw-emr-lab-2026/logs/
```

This single lab demonstrates **Primary/Core/Task architecture + cluster sizing + PySpark processing + EMR Managed Scaling + S3 integration** end to end.